Chapter 2 | A Simple Compiler

考場找答案：token / RE → 本頁；CFG / predict / parser → p.2；推導 / parse tree / AST → p.3；symbol table / type / error → p.4；dc code → p.5；課本習題 / scanner 擴充 → p.6。
題型依據：第一次小考的 Explain / Difference / 情境判斷。以下英文問答為**仿題型練習答案**，不是教師公布的第二次考題。

1 | ac 編譯器：每一站在處理什麼？

PPT 2、12、15、27–38

階段	輸入 → 輸出	關鍵工作
Scanner / lexer	Characters → tokens	依 regular expression 辨認詞法單位，略過 blank。
Parser	Tokens → AST	依 CFG 檢查語法；本章用預測式 recursive descent。
Symbol-table constructor	AST 宣告 → symbol table	記錄 identifier 的型別，檢查重複宣告。
Type checker	AST + symbol table → decorated AST	查宣告、由葉往根求型別，必要時加入 int2float。
Code generator	Checked AST → dc code	走訪節點；先產生運算元，再產生運算子。

短答：The scanner groups characters into tokens. The parser checks their grammatical structure and builds an AST. Semantic analysis checks declarations and types; code generation translates the checked AST into dc instructions.

2 | Token：kind 與實際字串

PPT 3–5、12–14

Explain what a token / lexeme is.

A token is a lexical unit with a type and, when needed, a value. Its lexeme is the actual character sequence in the source.

`b`：type = `id`，value = `"b"`。`3.2`：type = `fnum`，value = `"3.2"`。Parser 選規則看 **type**，符號表 / 產碼需要 **value**。

Token	ac 規則 / lexeme
floatdcl / intdcl / print	`f / i / p`
id	`[a-eghj-oq-z]`，單一字母
assign / plus / minus	`= / + / -`
inum	`[0-9]+`
fnum	`[0-9]+\.[0-9]+`
blank	原表為一個以上空格；略過
$	輸入結尾；不是要打的字元

RE 的 `+` 是一次以上；`\.` 是字面小數點。原圖未跳脫，此處消除歧義。多行程式另需略過換行。

ac ≠ C：id 只有一個小寫字母，f、i、p 是保留字；沒有分號、括號、乘除或 `//` 註解。投影片的註解是給人看的。

3 | 切 token：短例快查

輸入	Scanner 的結果
`b2`	id(b), inum(2)；不是單一 id
`abc`	id(a), id(b), id(c)
`007`	一個 inum(007)
`-5`	minus, inum(5)；負號非數字一部分
`.5`	首字元「.」無 token → lexical error
`5.`	Fig 2.3 不合法；Fig 2.6 虛擬碼會收

4 | RE 與 CFG 的分工

What is the difference between a regular expression and a context-free grammar?

In this compiler, regular expressions specify the character patterns of tokens. A context-free grammar specifies how tokens form declarations, expressions, statements, and programs.

terminal：不能再改寫，本章是 token kinds。
non-terminal：可依 production 改寫。
production：`A → α`，將 LHS 的 A 換成 RHS 的 α。
start symbol：本章為 Prog。
λ / ε：空字串，不消耗任何 token。
$：end-of-input marker，與 λ 不同。

Why are scanning and parsing separate?

Scanning handles character-level patterns; parsing handles token-level structure. This separation simplifies the grammar and lets the parser operate on token types.

5 | 課本範例：一路都用它

PPT 5

```
f b
i a
a = 5
b = a + 3.2
p b
```

```
floatdcl id | intdcl id | id assign inum
id assign id plus fnum | print id | $
```

共 14 個來源 tokens（| 僅分句）；再加 scanner 回傳的 `$` 共 15 個。空白不計。宣告 b:float、a:integer；最後印出 8.2。

6 | Scanner 作答骨架

略過 blank → EOF 回 $ → 數字呼叫 SCANDIGITS → 否則吃一字元分類 → 未定義字元報 lexical error。

PEEK：看但不吃；**ADVANCE**：吃掉並前進。Scanner 的 lookahead 是**字元**；parser 的 lookahead 是 **token**。

5. 的答法：問 formal definition → 不合法；問「照 Fig 2.6 執行」→ fnum，因小數點後的 while 可跑 0 次。

文法 × Predict Set × Parser

7｜文法與每條產生式的 predict set

PPT 7、17–24；全表依文法推算

#	Production	Predict set	選它時做什麼
1	`Prog → Dcls Stmts $`	{floatdcl, intdcl, id, print, $}	DCLS(); STMTS(); MATCH($)
2	`Dcls → Dcl Dcls`	{floatdcl, intdcl}	剖析一個宣告，再繼續
3	`Dcls → λ`	{id, print, $}	宣告結束，不吃 token
4	`Dcl → floatdcl id`	{floatdcl}	MATCH(floatdcl); MATCH(id)
5	`Dcl → intdcl id`	{intdcl}	MATCH(intdcl); MATCH(id)
6	`Stmts → Stmt Stmts`	{id, print}	剖析一個敘述，再繼續
7	`Stmts → λ`	{$}	敘述結束，不吃 token
8	`Stmt → id assign Val Expr`	{id}	指定敘述
9	`Stmt → print id`	{print}	只能印變數，不能 p 5 或 p a+1
10	`Expr → plus Val Expr`	{plus}	繼續處理「+ 值」
11	`Expr → minus Val Expr`	{minus}	繼續處理「− 值」
12	`Expr → λ`	{id, print, $}	算式結束；lookahead 留給下一句
13	`Val → id`	{id}	變數
14	`Val → inum`	{inum}	整數常數
15	`Val → fnum`	{fnum}	浮點常數

7 個 non-terminals：Prog, Dcls, Dcl, Stmts, Stmt, Expr, Val。**10 個 terminals**：floatdcl, intdcl, print, id, assign, plus, minus, inum, fnum, $。Blank 已略過，λ 不是 terminal。

> **文法限制**：宣告在全部敘述前；Val 不接受負號 / 括號。`a=5` 是 **Val(5) + Expr(λ)**，Expr 只是「後續 ± Val 串」。Dcls、Stmts 都可為空，所以空輸入在此 CFG 中合法。

8｜Predict 怎麼算、怎麼解釋？

What is a predict set? Why must the sets be disjoint?

A predict set contains the lookahead tokens for selecting a production. For predictive parsing with one token of lookahead, alternatives of the same nonterminal must have disjoint predict sets so that the choice is unique.

1. RHS 第一個是 terminal t → {t}。
2. 第一個是 non-terminal → 找它能導出的第一個 token；若可為空，繼續看後面符號。
3. 整個 RHS 可為 λ → 加入 LHS 後面能接的 token。

> **補充公式（FIRST / FOLLOW）**：
> Predict(A → α) = FIRST(α) − {λ}；
> 若 α ⇒* λ，再聯集 FOLLOW(A)。
> 這是推算工具，Ch2 先用口語說明。

Stmts → Stmt Stmts：Stmt 可由 id 或 print 開始，故 {id, print}。
Stmts → λ：Prog 裡 Stmts 後面是 $，故 {$}。
Expr → λ：後面是下一個 Stmt 或程式結尾，故 {id, print, $}。

> λ 分支**不能**寫「前面分支都不是就 return」。Expr 遇到 inum 並不合法；λ 不表示吞掉未知輸入。

9｜Recursive descent 的程序模板

Recursive descent uses one procedure per nonterminal. It selects a production using lookahead, matches terminals, and calls procedures for nonterminals in RHS order.

```
教材虛擬碼整理（不建 AST 的辨識版）
STMT():
  if peek() == id:
    match(id); match(assign)
    VAL(); EXPR()
  else if peek() == print:
    match(print); match(id)
  else: error()

STMTS():
  if peek() in {id, print}:
    STMT(); STMTS()
  else if peek() == $: return   // λ
  else: error()
```

MATCH(t)：確認 lookahead 為 t，成功才消耗；不符即 syntax error。**LL(1)**：由左到右讀輸入（L），最左推導（L），看 1 個 token。

What if one lookahead cannot select a unique rule?

Rewrite the grammar or use more lookahead, if sufficient. Overlapping predict sets prevent a unique LL(1) choice; they do not by themselves prove ambiguity.

Recursive descent 也可能使用回溯；本章採 predict set 選分支。不能推論所有文法都可藉增加 lookahead 解決。

Derivation × Parse Tree × AST

10｜三種表示，回答的問題不同

PPT 8–11、27–28

	Derivation 推導	Parse tree 剖析樹	AST 抽象語法樹
重點	依序用了哪些 productions	文法規則如何形成階層	程式操作與運算元
保留	每一步 sentential form	non-terminals、terminals、λ、$	宣告、assign、plus/minus、print、值
省略	—	獨立子樹的改寫先後	Dcls/Stmts/Val/Expr 包裝、λ、$

Difference：A parse tree records the full grammatical structure of a derivation. An AST omits grammar-only details and keeps the operations and operands needed by later compiler phases.

最左推導：每次展開最左 non-terminal；**最右推導：**每次展開最右 non-terminal。同一棵 parse tree 的最左與最右推導只差展開順序，不因此變成兩棵不同的樹。

11｜完整最左推導：i a a=5 p a

```
Prog
⇒ Dcls Stmts $                          (1)
⇒ Dcl Dcls Stmts $                      (2)
⇒ intdcl id Dcls Stmts $                (5)
⇒ intdcl id Stmts $                     (3)
⇒ intdcl id Stmt Stmts $                (6)
⇒ intdcl id id assign Val Expr Stmts $ (8)
⇒ intdcl id id assign inum Expr Stmts $ (14)
⇒ intdcl id id assign inum Stmts $    (12)
⇒ intdcl id id assign inum Stmt Stmts $ (6)
⇒ intdcl id id assign inum print id Stmts $ (9)
⇒ intdcl id id assign inum print id $  (7)
```

共 11 次改寫，12 個句型。規則序列：`1,2,5,3,6,8,14,12,6,9,7`。

最後是 token kinds；a、5 是 attributes。考題要「程式字串」時，可在 token 葉旁註記 lexeme。

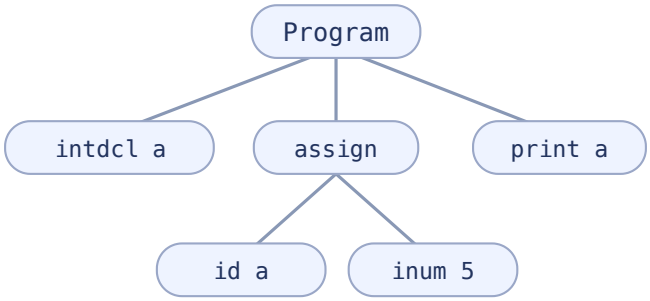
12｜畫 parse tree 的檢查法

1. 根為 Prog，孩子 Dcls、Stmts、$。
2. 每次展開完整照 RHS 左到右放孩子。
3. Dcls / Stmts 遞迴串接，最後 λ 收尾。
4. 指定式：id、assign、Val、Expr。
5. 葉由左到右忽略 λ，等於完整 token 串。

不要提前變成 AST：parse tree 的 plus 是 Expr 底下的 terminal 葉；AST 的 plus 才是兩個運算元的父節點。嚴格文法樹中 Val 下保留 inum / id 這層。

13｜同一例的 parse tree 與 AST

```
Parse tree（括號為 lexeme 註記）
Prog
├─ Dcls
│  ├─ Dcl
│  │  ├─ intdcl (i)
│  │  └─ id (a)
│  └─ Dcls → λ
├─ Stmts
│  ├─ Stmt
│  │  ├─ id (a)
│  │  ├─ assign (=)
│  │  ├─ Val → inum (5)
│  │  └─ Expr → λ
│  └─ Stmts
│     ├─ Stmt
│     │  ├─ print (p)
│     │  └─ id (a)
│     └─ Stmts → λ
└─ $
```



AST 重新繪製，保留語意結構；不是投影片原圖。

14｜遞迴方向不等於運算結合性

PPT 28、42；補充辨析

本課習題圖：`x+30+1` 畫成 `plus(x, plus(30,1))`（右側較深）。要求依教材圖畫，就照該形狀。
一般左結合語意 / 本地 C 實作：`minus(minus(x,30),1)` 表示 `(x-30)-1`。x=100 時得 69；`x-(30-1)` 得 71。未指定時說明採用的結合規則，不能只憑右遞迴斷定語意。

A right-recursive grammar does not force a left-associative language to use a right-associated AST. Semantic actions can accumulate the left operand to construct a left-associated tree.

本章整數加法例兩種形狀都得 131；不能泛化成任何數值語意下都可任意重排加法。

Symbol Table × Type × Errors

15 | Symbol table

PPT 29–33

What does the symbol table store? Why 23 entries?

The ac symbol table maps each identifier to integer, float, or unused (null). There are 23 possible identifiers: the 26 lowercase letters excluding the reserved letters f, i, and p.

動作	規則
初始化	全部設 null（尚未宣告）
EnterSymbol(name,type)	null 才填入；已有型別 → duplicate declaration
LookupSymbol(name)	回傳型別；null 代表未宣告
範例 f b / i a	b→float、a→integer；其他 21 項 null

null 是 unused 狀態，**不是第三種運算型別**。表記錄型別，不是 a=5、b=8.2 這些執行期值。C 補充實作用 26 格陣列方便索引，課本仍是 23 個可用名稱。

16 | Type checking 為何 bottom-up？

The type of a compound expression depends on its operands. Therefore, type checking processes the children before the parent, looking up identifier types and inserting conversions when needed.

節點	規則 / 依據
inum / fnum	integer / float（p.034）
id	查表；須已宣告（p.033）
plus / minus	依子節點型別決定自己型別
assign	RHS 與目標 id 型別相容
int2float	編譯器加入（Fig 2.13）

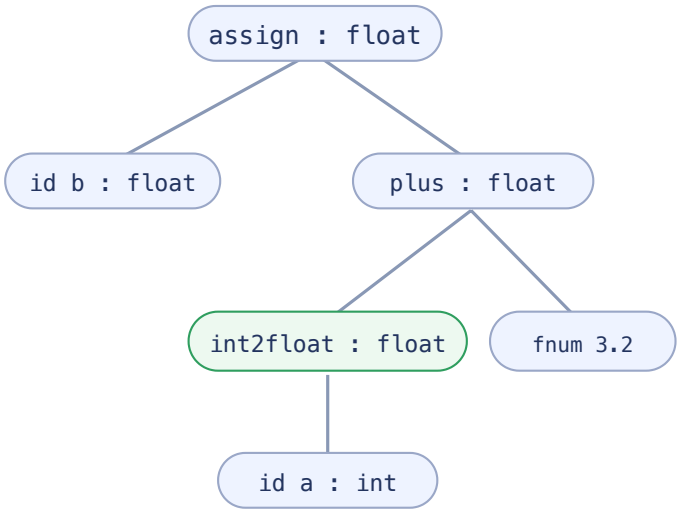
來源界限：p.034 的 utility methods（Fig 2.12）未附。Fig 2.13 確實把 `a+3.2` 的 a 包成 int2float。下面完整規則依此反推、由本地實作採用，**不是逐字教材規格**。

補充規則	結果
int ± int	int
int ± float / float ± int	int 那側轉 float，結果 float
float ± float	float
float ← int	RHS 外包 int2float
int ← float	拒絕隱式窄化（本地實作）

17 | Decorated AST：轉換插在哪？

PPT 35

`f b i a a=5 b=a+3.2 p b` 中第二個指定：



保留 a 的宣告：a 仍是 integer，只在這次運算使用其值時轉成 float。不是把 symbol table 的 a 改成 float。

A decorated AST carries semantic information such as types and may contain inserted conversion nodes. Here, int2float converts the value of a before floating-point addition.

18 | Semantic action：執行或記錄？

PPT 25–27

What happens after parsing a = 5?

An interpreter may execute the assignment immediately. In this chapter's compiler, the semantic action records the assignment in an AST for later analysis and code generation.

走完 `id assign Val Expr` 已知道目標和 RHS，可建 assign 節點。**本章**先建 AST，再檢查、產碼；不是所有 compiler 都必須如此排程。

19 | 情境題：Lexical / Syntax / Semantic？

依 ac 規則自編練習

輸入（各列獨立）	第一個問題	可直接寫的英文理由
`i a a=@`	Lexical	The character @ does not match any ac token.
`i a a=+5`	Syntax	Val cannot begin with plus; ac has no unary plus here.
`i a p 5`	Syntax	The print production requires an identifier, not a number.
`i a a=5 f b`	Syntax	Declarations cannot occur after statements.
`i a f a`	Semantic	The identifier a is declared twice.
`f b b=c`	Semantic	The identifier c has not been declared.
`i a a=3.2`	Semantic（補充型別規則）	A float value cannot be implicitly assigned to an integer variable.

通用答法：A syntax error violates the grammar. A static semantic error violates declaration or type rules even though the token sequence is grammatically valid.
合法程式算出非預期答案可能是 logic error；編譯器不一定知道人的意圖。未宣告與未初始化也是兩個不同問題。

Code Generation | dc 堆疊機

20 | 六種 visitor：寫出碼的模板

PPT 36–38 · Fig 2.14–2.15

AST 節點	依序輸出的碼	意義
Consting：inum / fnum	`value`	常數推入堆疊
SymReferencing：id r	`lr`	複製暫存器 r 的值到堆疊
Computing：L op R	`code(L); code(R); op`	先左、再右、最後運算子
Converting：int2float(E)	`code(E); 5 k`	教材以設定 scale 表達轉換
Assigning：r = E	`code(E); sr; 0 k`	彈出 RHS 結果存入 r，重設 scale
Printing：p r	`lr; p; si`	取值、印值、彈出丟到暫存器 i
宣告	無	只影響符號表，不輸出 dc 指令

分號僅分隔「依序輸出」的指令，不是 dc 字元。`lr/sr` 的 r 是變數名，例如 `la/sa`。

Why postfix? A stack machine needs the operands before the operator. The code generator therefore emits code for the left child, then the right child, and finally the operation.
走訪從根呼叫，不代表先輸出根的運算子；計算節點實際 emit 順序是後序（postorder）。

21 | 完整例：b = a + 3.2

```
ac source       dc target
f b             (宣告無碼)
i a             (宣告無碼)
a = 5           5
                sa
                0 k
b = a + 3.2     la
                5 k
                3.2
                +
                sb
                0 k
p b             lb
                p
                si
```

轉換位置： `int2float(id a)` → 先 `la` 再 `5 k`。不是在整個加法後才放轉換。

22 | dc 指令的易混處

- `sr` 會 **pop**；`lr` 只讀暫存器、push 副本。
- `p` 印出 top，**不 pop**；用 `si` 清理。
- i 是 ac 保留字、不能當變數名，作 dc 垃圾暫存器不會覆蓋使用者變數。
- `5 k`：push 5，再由 k pop 5 作 scale；不是把原有 top 改成 5。
- `0 k` 不把已存入暫存器的 8.2 變成 8。

教材模型與 dc 行為： `k` 設定 scale，並非硬體的 int→float 指令。dc 的加減小數位由運算元決定，不能說 `5 k` 使所有輸出固定五位；考產碼仍照 Fig 2.14。

23 | 手算 stack：右側是頂端

指令	執行後 stack	其他狀態
`5`	[5]	—
`sa`	[]	a = 5
`0 k`	[]	scale = 0
`la`	[5]	a 保持 5
`5 k`	[5]	scale = 5
`3.2`	[5, 3.2]	—
`+`	[8.2]	5 + 3.2
`sb`	[]	b = 8.2
`0 k`	[]	scale = 0
`lb`	[8.2]	—
`p`	[8.2]	印出 8.2
`si`	[]	i = 8.2（丟棄用）

24 | 減法：一定要分左右

`L R -` 計算 **L − R**；先 pop 的是右運算元 R，第二個 pop 才是 L。

```
左結合：(100 - 30) - 1
100 30 - 1 -        → 69

右結合：100 - (30 - 1)
100 30 1 - -        → 71
```

`x=x-30` 產碼：`lx 30 - sx 0 k`。不是 `30 lx -`，也不應先 `sx` 才計算 RHS。

Why is si emitted after p?

The dc print command leaves the value on the stack. The generated si pops it into the unused register i, restoring the stack for subsequent statements.

教材習題 × Scanner 擴充

25 | HW 1–2 PPT 39–44

題目： `i x x=100 x=x+30+1 p x`

HW 1：parse tree / 最左推導

按 p.2 文法，規則序列為：

`1,2,5,3,6,8,14,12,6,8,13,10,14,10,14,12,6,9,7`（19 次改寫）。

第二句指定的尾巴：

`Expr → plus Val Expr`，Val→inum(30)；下一層 `Expr → plus Val Expr`，Val→inum(1)；最後 Expr→λ。

HW 2(a)：教材 AST 的結構

```
Prog
├─ intdcl x        // 原圖只標 x
├─ assign(id x, inum 100)
├─ assign
│  ├─ id x
│  └─ plus
│     ├─ id x
│     └─ plus
│        ├─ inum 30
│        └─ inum 1
└─ print x
```

補齊節點種類；樹形保留 p.042 的右結合。

HW 2(b)：語意正確嗎？

教材解答 p.043：先答「No」，接著說 AST 的 30、+、1 可合成單一節點 31。

待核對：p.033–034 所列宣告 / 型別規則未顯示此例違規。把常數子樹算成 31 是 **constant folding（常數摺疊）**，不等於修正語意錯誤。問「依投影片解答」時保留 No 及說明；問規則推理時明列 x 已宣告、運算元皆 integer。不可捏造「ac 不准連續加法」。

HW 2(c)：教材摺疊後的 dc code

```
100 sx 0 k
lx 31 + sx 0 k
lx p si
```

印出 **131**。原右結合 AST 未摺疊的第二行：`lx 30 1 + + sx 0 k`。本地 C 左結合版：`lx 30 + 1 + sx 0 k`。本例結果相同，形狀與碼序不同。

26 | HW 3：擴充 scanner PPT 45–46

(a)(b) 宣告可用完整單字

```
floatdcl  "f" | "float"
intdcl    "i" | "int"
```

程式也要改：看到 f/i 時不能立刻只回一個字元；檢查完整關鍵字是否匹配，未匹配字元保留給後續 token。明定最長匹配 / 關鍵字邊界規則。

原 ac 的 `fl` 是 `floatdcl id(l)`，仍須辨認。完整 `float` 匹配時選長 token，不能拆成 f、l、o、a、t。

(c) 可選的有號指數

教材原解答（照錄結構）：

```
inum  [0-9]+ e -? [0-9]+
fnum  [0-9] . [0-9]+ e -? [0-9]+
```

與題意有三個落差：① 指數變必填；② 只收負號、漏正號；③ fnum 小數點前只有一位。「依題意擴充」時應補齊。

依題意補充：接受原數字及指數形式；帶指數者歸 fnum（`123e-22` 不一定是整數）。

```
inum  [0-9]+
fnum  [0-9]+\.[0-9]+(e[+-]?[0-9]+)?
      | [0-9]+e[+-]?[0-9]+
```

`e` 依題目小寫；要 E 另明定 `[eE]`。流程：數字 → 可選小數部分 → 可選 e → 可選 +/− → **至少一位指數數字**。

應接受（補充規則）	非完整數字格式
5、3.2、1.0e10 123e-22、123.4e+5	1e、1e+、3.e2、.5

不完整指數依明定 scanner 規則報 malformed number，或退回較短 token 由後續階段拒絕；RE 本身未決定所有錯誤回復策略。

Extension answer：Update both the token patterns and the scanner. Recognize the optional exponent and its sign, require exponent digits, and preserve characters that belong to the next token.

後端：dc 不直接接受科學記號；須轉成可輸出的十進位寫法，不能直接 EMIT `1e10`。

27 | 最後 30 秒：英文短答骨架

Explain X：definition → role / input–output → tiny example。
Difference：“X checks / represents …, whereas Y checks / represents ….”
Is it valid?：“Lexically …; syntactically …; semantically …, because ….”
How to extend?：token rule → scanner behavior → parser/type/codegen 是否受影響。

來源與範圍：Ch02 a simple compiler.pptx / 46 頁衍生 PDF、四份逐頁伴讀（p.001–046）；題型參考「Compiler – 第一次小考題目與標準答案」，版面參考 Ch1 cheatsheet。6 頁 A4（雙面 3 張），未假設允許的頁數上限。
參照：Chapter 2 原投影片 PDF · 第一次小考 · GNU dc manual。
待核對保留：p.034 缺 Fig 2.12；p.043 的 No 與既列規則不一致。p.014、p.042、p.046 的定義 / 樹形 / 解答差異已明示。